Algorithmic Problem Solving 2026

Project Report

Group	Group H
Members	Karl Thedor Ruby (krub@itu.dk) Kristoffer Mejborn Eliasson (krme@itu.dk) Lukas Shaghashvili-Johannessen (lush@itu.dk)
Supervisor	Holger Dell, Martin Aumüller
Course	Algorithmic Problem Solving (APS 2026)
Date	[YYYY-MM-DD]

IT University of Copenhagen

Contents

1. Abstract

Provide a short summary (max 200 words) describing the designed problem, the solved problems, and the main algorithmic techniques used.

2. Designed Problem — The Siege

2.1. Problem description

You are Warchief Grukk, commanding an assault on N dwarven strongholds. Each stronghold is a network of one-way mine shafts: carts can only travel in one direction through each tunnel (the network is a directed graph). Deep inside each stronghold is the Gold Vein (room 0, the source); the entrance where wolf-riders wait is room $V_i - 1$ (the sink). Each tunnel has a weight limit (capacity) giving the maximum kilograms of gold carts can carry through it per hour. Your miners extract gold at the maximum possible rate — the max-flow of the network.

Each stronghold costs c_i orc warriors to conquer. You have B warriors total. Choose which strongholds to conquer to maximise total gold extracted.

Input. First line: integers N and B ($1 \leq N \leq 50, 1 \leq B \leq 1000$). Then N stronghold descriptions, each starting with: name s_i (lowercase, length ≤ 20), cost c_i ($1 \leq c_i \leq B$), rooms V_i ($2 \leq V_i \leq 15$), edges E_i ($1 \leq E_i \leq 40$). Followed by E_i lines each giving u, v, w for a directed tunnel from room u to v with capacity w ($1 \leq w \leq 100$).

Output. One line per selected stronghold, format `<index> <name>`, in ascending index order. The optimal subset is guaranteed unique.

Sample input 1:

```
3 10
ironkeep 3 3 2
0 1 5
1 2 2
stonewall 6 4 4
0 1 3
0 2 2
1 3 2
2 3 3
thornhold 4 3 2
0 1 4
1 2 3
```

Sample output 1:

```
1 stonewall
2 thornhold
```

Max-flows: ironkeep = 2 (bottleneck edge 1⟶2, cap 2), stonewall = 4 (two augmenting paths: 0⟶1⟶3 flow 2, then 0⟶2⟶3 flow 2), thornhold = 3. With $B = 10$: stonewall+thornhold costs $6 + 4 = 10 \leq 10$, gold $4 + 3 = 7$. ironkeep+stonewall costs 9, gold $2 + 4 = 6$. So optimal is stonewall+thornhold.

2.2. Intended solution

The problem decomposes into two independent subproblems applied sequentially:

1. **Per-stronghold max-flow.** For each stronghold i, compute the maximum flow from room 0 to room $V_i - 1$ using Edmonds-Karp (BFS-augmented Ford-Fulkerson). This gives the gold value g_i.

2. **0/1 Knapsack DP.** Treat each stronghold as a knapsack item with weight c_i and value g_i. Find the subset of items with total weight $\leq B$ that maximises total value.

```
function solve():
    for each stronghold i:
        g[i] = edmonds_karp(graph_i, source=0, sink=V_i - 1)

    dp[0..N][0..B] = 0              // dp[i][j] = best gold using first i
strongholds, budget j
    for i in 1..N:
        for j in 0..B:
            dp[i][j] = dp[i-1][j]    // skip stronghold i
            if j >= c[i]:
                dp[i][j] = max(dp[i][j], g[i] + dp[i-1][j - c[i]])

    // backtrack to recover selected indices
    remaining = B
    for i from N down to 1:
        if dp[i][remaining] != dp[i-1][remaining]:
            select stronghold i-1
            remaining -= c[i-1]
    output selected in ascending order
```

Time complexity:
- Edmonds-Karp per stronghold: $O(V_i \cdot E_i^2)$. With $V_i \leq 15$, $E_i \leq 40$: at most $15 \times 40^2 = 24000$ operations each.
- All N max-flows: $O(N \cdot V_{\max} \cdot E_{\max}^2) \approx 50 \times 24000 = 1.2 \times 10^6$.
- Knapsack DP: $O(N \cdot B) = 50 \times 1000 = 50000$.
- **Overall:** $O(N \cdot VE^2 + NB)$ — effectively constant given the small bounds.

Memory: $O(V_{\max}^2)$ for the adjacency matrix per flow computation, $O(N \cdot B)$ for the DP table.

2.3. Test-case design

Sample tests (manually crafted, small enough for hand-verification):

- **1-small-manual:** 3 strongholds, $B = 10$. Tests basic knapsack selection; greedy by ratio picks the wrong pair.
- **2-medium-manual:** 3 strongholds with parallel-path graphs. Verifies that multi-path flow is summed correctly.
- **3-single-manual:** $N = 1$, tests single-stronghold boundary; the one stronghold is within budget so it is selected.
- **4-zeroflow-manual:** One stronghold has no s-t path (disconnected graph, flow = 0). Tests that a zero-gold stronghold is correctly skipped when a positive-gold alternative exists within budget.

Secret tests (generated by `generators/generator.py` or hand-crafted for specific corner cases):

- **1-single:** $N = 1$, B large — single item always selected if affordable.
- **2-small:** $N \leq 5$, $B \leq 20$ — brute-force can verify correctness.
- **3-medium:** $N \approx 20$, $B \approx 200$ — exercises DP without hitting any limit.
- **4-large:** $N = 50$, $B = 1000$ — maximum constraint stress test; brute-force 2^{50} is impossible.
- **5-tiebreak:** Constructed so two subsets tie in gold before one is broken by careful cost arrangement; validator enforces uniqueness, so this test confirms no two optimal subsets exist.

- **6-zeroflow:** Multiple strongholds with disconnected graphs (no s-t path, gold = 0); tests that zero-value items do not inflate the knapsack answer.
- **7-cycle:** Graphs containing cycles and back-edges; confirms that residual graph reverse-edge handling in Edmonds-Karp is correct (a bug here would under-count flow).

Corner cases covered: $N = 1$; B exactly equal to one item cost; all items affordable; no item affordable; zero flow; graphs that are chains, parallel paths, or contain cycles.

2.4. Accepted and wrong solutions

Accepted — `submissions/accepted/solution.py` (Python 3) and `solution.rs` (Rust): Edmonds-Karp max-flow per stronghold, then 0/1 knapsack DP with backtracking. Both produce identical output. The Python solution runs comfortably within 2 seconds given the small problem bounds; the Rust solution is included as a fast reference.

TLE — `submissions/time_limit_exceeded/brute_force.py` / `brute_force.rs`: Enumerates all 2^N subsets, computes total cost and gold for each, and keeps the best feasible subset. Correct for small N but $2^{50} \approx 10^{15}$ iterations makes it infeasible for $N = 50$. Used to verify correctness of the DP solution on small inputs.

WA — `submissions/wrong_answer/greedy.py`: Sorts strongholds by the gold-per-warrior ratio $\frac{g_i}{c_i}$ descending and greedily adds items as long as they fit in the remaining budget. This is the optimal strategy for **fractional** knapsack but fails for 0/1 knapsack. Counter-example from sample 1: ratios are stonewall ≈ 0.67, thornhold 0.75, ironkeep ≈ 0.67. Greedy picks thornhold ($c = 4$, $g = 3$) then ironkeep ($c = 3$, $g = 2$), total cost 7, gold 5. Optimal is stonewall + thornhold ($c = 10$, $g = 7$).

2.5. Input validation & formatting

`input_validators/validator.py` enforces all constraints via regex and range checks, then runs a full knapsack over all test inputs to confirm the optimal solution is **unique** (exits with code 43 if ≥ 2 optimal subsets exist, exits with code 42 on success). Specific checks:

- Line 1 matches `^[1-9][0-9]* [1-9][0-9]*\n$`; $N \in [1, 50]$, $B \in [1, 1000]$.
- Each castle header: name lowercase letters only, length ≤ 20, globally unique; $c_i \in [1, B]$; $V_i \in [2, 15]$; $E_i \in [1, 40]$.
- Each edge line: $u, v \in [0, V_i - 1]$, $u \neq v$, no duplicate (u, v) pair within the same stronghold; $w \in [1, 100]$.
- No trailing content after the last edge.

Contestants may assume all input is well-formed and within these bounds.

2.6. Verify notes

`verifyproblem` was run from the `problem/thesiege/` directory. All sample and secret test cases pass the validator (exit 42). The accepted solutions pass all tests. The brute-force solution is correctly flagged TLE on `4-large` and the greedy is correctly flagged WA on `1-small-manual`. No warnings were produced.

2.7. Testing and Validation

Run the accepted Python solution on a sample test:

```
python3 submissions/accepted/solution.py < data/sample/1-small-manual.in
```

Run the validator on an input (exit code 42 = valid and unique-optimal, 43 = invalid):

```
python3 input_validators/validator.py < data/sample/1-small-manual.in; echo "exit $?"
```

Generate a large random test and time the accepted solution:

```
python3 generators/generator.py 42 50 1000 15 40 100 > /tmp/large.in
time python3 submissions/accepted/solution.py < /tmp/large.in
```

Diff DP against brute-force on a small instance:

```
python3 generators/generator.py 7 5 20 8 15 50 > /tmp/small.in
python3 submissions/accepted/solution.py < /tmp/small.in > /tmp/dp.out
python3 submissions/time_limit_exceeded/brute_force.py < /tmp/small.in > /tmp/bf.out
diff /tmp/dp.out /tmp/bf.out
```

2.8. Source for Designed Problem

Repository layout under `problem/thesiege/`:

```
problem/thesiege/
  problem.yaml                       problem metadata (name, type, uuid)
  problem_statement/problem.en.tex   LaTeX problem statement
  submissions/
    accepted/solution.py             Python 3 accepted solution
    accepted/solution.rs             Rust accepted solution
    time_limit_exceeded/brute_force.py
    time_limit_exceeded/brute_force.rs
    wrong_answer/greedy.py
  generators/generator.py            parametric random generator
  input_validators/validator.py      constraint + uniqueness checker
  data/
    sample/   4 hand-crafted sample cases (.in / .ans)
    secret/   7 generated/hand-crafted secret cases (.in / .ans)
```

Run locally with `verifyproblem` (requires the `problemtools` package):

```
cd problem/
verifyproblem thesiege/
```

3. Solved Problems

3.1. Robert Hood (Geometric Algorithms)

3.1.1. Problem metadata

- **Problem name / Kattis link:** Robert Hood
- **Short formal I/O description:** Given n points in the plane with integer coordinates, output the maximum Euclidean distance between any two points, as a floating-point number.

3.1.2. Solution overview

The maximum distance between any two points in a finite set is always achieved by two points on the convex hull of the set. This reduces the problem to two steps: compute the convex hull, then find the diameter of the hull polygon.

The convex hull is computed using Andrew's monotone chain algorithm, which sorts the points lexicographically and constructs the lower and upper hull in two linear passes. The result is the vertices of the hull in counter-clockwise order.

The diameter of the convex polygon is then found using the rotating calipers technique. For each edge of the hull, the algorithm advances an antipodal pointer j as long as the next vertex is farther from the edge than the current one. The cross product of the edge direction and the vector between consecutive candidate points determines the direction to move j. Both endpoints of the current edge are checked against j at each step, so no antipodal pair is missed.

This approach was chosen because it achieves optimal asymptotic complexity and avoids the $O(m^2)$ brute-force over hull vertices, which could be slow when all input points lie on the hull.

3.1.3. Correctness argument

Reduction to hull. Any point strictly inside the convex hull cannot be a diameter endpoint, since projecting it outward to the hull boundary only increases distance. Therefore the maximum distance is realised by two hull vertices.

Rotating calipers. The algorithm maintains the invariant that j is the index of the vertex farthest from the directed edge between `hull[i]` and `hull[i+1]`, among all indices not yet "passed" by i. The cross product check `cx > 0` advances j when the next vertex is strictly farther, and stops otherwise. Because the hull is convex, the distance from an edge is unimodal over consecutive vertices, so the pointer j never needs to retreat. Both `hull[i]` and `hull[(i+1) % m]` are compared to `hull[j]` at each step to cover all antipodal pairs correctly.

Edge cases. When $m = 1$ (all points identical) the distance is 0. When $m = 2$ (collinear or just two distinct points) the distance is computed directly. These are handled before the calipers loop.

3.1.4. Complexity and time-limit reasoning

Let n be the number of input points and m the number of hull vertices, with $m \leq n$.

- **Convex hull:** $O(n \log n)$ dominated by the initial sort.
- **Rotating calipers:** $O(m)$ since each pointer advances at most m times.
- **Overall:** $O(n \log n)$ time and $O(n)$ space.

For $n = 10^5$, this is approximately $10^5 \times 17 \approx 1.7 \times 10^6$ operations for the sort, plus a linear pass. The implementation avoids square roots until the final step by comparing squared distances, eliminating floating-point overhead from the inner loop.

3.1.5. Empirical worst-case testing

The worst case for the rotating calipers occurs when all input points lie on the convex hull, maximising m. A set of n points uniformly distributed on a large circle achieves this. The following generator was used:

```
import math
N = 100000
print(N)
for i in range(N):
    angle = 2 * math.pi * i / N
    x = round(1e9 * math.cos(angle))
    y = round(1e9 * math.sin(angle))
    print(x, y)
```

Running this input on the solution completed in approximately 0.73 seconds in CPython 3, which is within the Kattis time limit.

3.1.6. Implementation notes

- **Language used:** Python 3
- **Files:** `solutions/geometry/roberthood.py`
- **How to run:** `python3 solutions/geometry/roberthood.py < input.txt`

3.2. Cookie Selection (Algorithms on Arrays)

3.2.1. Problem metadata

- **Problem name / Kattis link:** Cookie Selection
- **Short formal I/O description:** Given a sequence of up to 2×10^5 events, each either inserting a positive integer into a multiset or querying and removing the element at rank $\lfloor \frac{n}{2} \rfloor + 1$ (1-indexed) from the sorted multiset of current size n, output each queried value on its own line.

3.2.2. Solution overview

The core challenge is maintaining a dynamic multiset that supports two operations in sublinear time: insertion of an element and extraction of the element at a given rank. A naive sorted list achieves $O(n)$ per operation, which is too slow for $n = 2 \times 10^5$.

The solution uses a Fenwick tree over coordinate-compressed values. Because cookie diameters are up to 10^9 but there are at most 2×10^5 distinct values in the input, all values are mapped to contiguous indices $1, 2, ..., m$ where m is the number of distinct values. The BIT stores counts: position i holds how many cookies with compressed value i are currently in the holding area. A prefix sum query then gives the number of cookies with value at most i.

To answer a rank query for rank k, the solution uses a binary descent on the BIT structure, finding the smallest index i such that the prefix sum up to i is at least k. This descent runs in $O(\log m)$ rather than $O(\log^2 m)$ compared to binary searching over prefix sum queries.

This approach was selected because the BIT offers simple, cache-friendly implementation while achieving the necessary $O(\log n)$ per operation.

3.2.3. Correctness argument

Coordinate compression. All values are read before processing begins. The mapping compress : $v \mapsto i + 1$ is a bijection from the set of distinct input values to $\{1, ..., m\}$, preserving order. Thus rank in the compressed BIT equals rank in the original multiset.

BIT invariant. After each insertion of value v, the BIT count at position compress(v) is incremented by one. After each removal, it is decremented by one. Therefore, at any point, `bit.query(i)` equals the number of cookies currently in the holding area with compressed index at most i, which equals the number of cookies with diameter at most decompress(i).

Rank formula. The problem specifies that for n cookies the cookie sent is at position $\lfloor \frac{n}{2} \rfloor + 1$ in sorted order for both odd and even n. Setting `rank = n // 2 + 1` before each removal matches this specification exactly.

kth correctness. The binary descent in `kth(k)` maintains the invariant that `prefix_sum[1..pos] < k` throughout the loop and terminates with the smallest `pos + 1` satisfying `prefix_sum[1..pos+1] >= k`, which is exactly the k-th occupied position.

3.2.4. Complexity and time-limit reasoning

Let N be the number of input lines and $m \leq N$ the number of distinct cookie diameters.

- **Preprocessing:** sorting the distinct values takes $O(N \log N)$.
- **Per event:** each insertion and each rank query performs one BIT update or descent, each costing $O(\log m) = O(\log N)$.
- **Overall:** $O(N \log N)$ time and $O(N)$ space.

For $N = 2 \times 10^5$ and a typical time limit of 1-2 seconds, roughly $2 \times 10^5 \times 17 \approx 3.4 \times 10^6$ elementary operations are required, which is well within budget even in Python given the constant factor of the BIT operations.

3.2.5. Empirical worst-case testing

A worst-case input alternates insertions and removals to keep the holding area non-empty throughout. The following generator was used:

```
N = 100000
for i in range(1, N + 1):
    print(i)
for _ in range(N):
    print('#')
```

This produces 2×10^5 lines: 10^5 insertions followed by 10^5 queries, exercising both the coordinate compression and the BIT at maximum size. Running this input on the solution completed in approximately 0.67 seconds in CPython 3, which is within the Kattis time limit.

3.2.6. Implementation notes

- **Language used:** Python 3
- **Files:** `solutions/arrays/cookieselection.py`
- **How to run:** `python3 solutions/arrays/cookieselection.py < input.txt`

3.3. Free Real Estate (Randomized Algorithms)

3.3.1. Problem metadata

- **Problem name / Kattis link:** Free Real Estate
- **Short formal I/O description:** Given an array of n integers all initialised to a, process q operations: point updates setting index i to value v, and range sum queries over $[s, e]$. For each query, output the sum or the string "it's free real estate" if the sum equals zero.

3.3.2. Solution overview

The problem requires point updates and range prefix-sum queries over a mutable array. A Fenwick tree (BIT) supports both operations in $O(\log n)$ time and $O(n)$ space, which is optimal for this problem structure.

Rather than storing the actual prices in the BIT, the solution stores only the **delta** from the initial value a. The BIT is initialised to all zeros. When penthouse i is updated to value v, the change `v - vals[i]` is added to the BIT at position i, and `vals[i]` is updated to v. For a range query $[s, e]$, the sum is computed as $(e - s + 1)c \cdot a + \text{BIT.query}(e) - \text{BIT.query}(s - 1)$, where the first term accounts for the initial uniform price and the BIT terms account for all accumulated deltas.

This representation avoids initialising the BIT with n values, keeping setup to $O(1)$ instead of $O(n \log n)$, which matters when n is large.

3.3.3. Correctness argument

Delta invariant. The BIT stores deltas from the initial value a, not absolute prices. After any sequence of updates, `BIT.query(j)` equals the sum of all changes applied to positions 1 through j.

Range sum. For a query $[s, e]$, the total price is the base contribution $(e - s + 1)c \cdot a$ plus the net deltas over that range, which is `BIT.query(e) - BIT.query(s-1)`. This matches the code exactly.

Update correctness. Each update computes `delta = v - vals[i]` before writing the new value to `vals[i]`. This means each delta is always relative to the actual current price, so repeated updates to the same index accumulate correctly in the BIT.

3.3.4. Complexity and time-limit reasoning

Let n be the array length and q the number of operations.

- **Initialisation:** $O(n)$ to allocate `vals`, $O(1)$ BIT setup.
- **Per update:** one BIT point update in $O(\log n)$.
- **Per query:** two BIT prefix queries in $O(\log n)$.
- **Overall:** $O(n + q \log n)$ time, $O(n)$ space.

For the largest test group ($n, q = 10^6$), this requires approximately $2 \times 10^6 \times 20 = 4 \times 10^7$ operations. The implementation uses `sys.stdin.buffer` for fast binary reads, avoiding the overhead of Python's line-by-line input parsing, which is critical at this scale.

3.3.5. Empirical worst-case testing

The worst case combines maximum array size with interleaved updates and queries over the full range. The following generator was used:

```
import random
N = 1000000
Q = 1000000
A = 1
print(N, Q, A)
for i in range(Q):
    if i % 2 == 0:
        idx = random.randint(1, N)
        v = random.randint(-10**9, 10**9)
        print(f'update {idx} {v}')
    else:
        s = random.randint(1, N)
        e = random.randint(s, min(s + 1000, N))
        print(f'query {s} {e}')
```

Running this input on the Kattis judge completed in approximately 0.48 seconds in CPython 3, well within the time limit.

3.3.6. Implementation notes

- **Language used:** Python 3
- **Files:** `solutions/random/freerealestate.py`
- **How to run:** `python3 solutions/random/freerealestate.py < input.txt`